

CodLab N°3: SanMarcosStore

App Android con Material Design 3 & Jetpack Compose @ Illarek-Lab

johnkbarrera.github.io | john.barrera@unmsm.edu.pe

1 Introduccion

Este laboratorio construye desde cero una aplicacion Android funcional denominada **SanMarcosStore**, integrando un tema personalizado generado con Material Theme Builder, componentes Material 3, navegacion entre pantallas y arquitectura modular. El objetivo es consolidar los conceptos teoricos de la Semana 3 en un producto ejecutable.

Axioma de Desarrollo Movil 2026

En el ecosistema Android moderno (2026), **Jetpack Compose** es el framework UI oficial y **Material Design 3** es el sistema de diseno obligatorio. La arquitectura recomendada es **Single-Activity Architecture**: una sola Activity que aloja multiples Composables como pantallas, con navegacion gestionada por NavHost.

2 Implementacion: SanMarcosStore

2.1 Requisitos Previos

Hardware & Software Checklist

En la estacion:	Android Studio Hedgehog 2023.1.1 o superior JDK 17 (incluido con Android Studio) Archivo <code>material-theme.zip</code> (Theme Builder) Conexion estable a internet para Gradle
En el dispositivo:	Emulador Pixel 7 con API 34 (recomendado) O dispositivo fisico con Android 7.0+ (API 24) Modo desarrollador activado (si es fisico)

2.2 Creacion del Proyecto en Android Studio

2.2.1 Seleccion del Template Correcto

Android Studio ofrece multiples templates con nombres similares. Solo uno es valido para Jetpack Compose.

Template	Usar	Razon
Empty Activity	✓	Compose limpio + MainActivity con tema vacio
Empty Views Activity	✗	XML viejo (sistema Views), no Compose

Trampa comun: si el template contiene la palabra **Views** en el nombre, el proyecto se generara con archivos XML legacy, incompatibles con el resto del laboratorio.

2.2.2 Configuración del Proyecto

Parámetros del Wizard de Creación

Name:	SanMarcosStore
Package name:	com.example.sanmarcosstore
Save location:	[elección del usuario]
Language:	Kotlin
Minimum SDK:	API 24 (Android 7.0 Nougat)
Build configuration:	Kotlin DSL (build.gradle.kts)

Tras el wizard, esperar 1-3 minutos al Gradle sync inicial.

2.2.3 Verificación de Jetpack Compose

Abrir `app/build.gradle.kts` y confirmar la presencia del bloque `buildFeatures`:

Confirmación de Compose Habilitado

```
android {
    // ...
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(platform("androidx.compose:compose-bom:..."))
    implementation("androidx.compose.material3:material3")
    implementation("androidx.activity:activity-compose:...")
}
```

Si estas líneas no están presentes, el template seleccionado fue incorrecto. Cerrar el proyecto y reiniciar desde el wizard.

2.3 Agregar Dependencias Necesarias

El template Empty Activity no incluye navegación, Google Fonts ni iconos extendidos. Se agregan manualmente al bloque `dependencies` de `app/build.gradle.kts`.

Dependencias del Laboratorio

```
dependencies {
    // ... dependencias por defecto ...

    // Activity Compose (version estable usada en clase)
    implementation("androidx.activity:activity-compose:1.10.0")

    // Tipografía descargable de Google Fonts
    implementation("androidx.compose.ui:ui-text-google-fonts:1.7.8")

    // Iconos extendidos de Material (Search, Favorite, etc.)
    implementation("androidx.compose.material:material-icons-extended:1.7.8")

    // Navegación entre pantallas
    implementation("androidx.navigation:navigation-compose:2.8.0")
}
```

Justificación de cada dependencia:

- `activity-compose`: integración entre Activity y Composables.
- `ui-text-google-fonts`: requerida por el `Type.kt` generado por Theme Builder.
- `material-icons-extended`: contiene más de 2,000 iconos. Sin esta dependencia:

- `Icons.Filled.Search`, `Icons.Filled.Favorite` y otros usados en el laboratorio aparecen en rojo.
- `navigation-compose`: para el `NavHost` y la navegacion entre las pantallas Tienda y Perfil.

Tras agregar las dependencias, click en **Sync Now** (banner amarillo).

2.4 Integracion del Tema (Material Theme Builder)

2.4.1 Estructura del ZIP Descargado

Contenido del material-theme.zip

```
material-theme/
+- README.md
+- ui/theme/
|   +- Color.kt           (paleta de colores)
|   +- Theme.kt           (configuracion del esquema)
|   +- Type.kt            (tipografia descargable)
+- res/values-v23/
|   +- font_certs.xml      (certificados Google Fonts)
```

2.4.2 Cambio de Package en los 3 Archivos del ZIP

El ZIP utiliza el package por defecto `com.example.compose`. Debe modificarse al package del proyecto antes de copiar los archivos.

Modificacion del package

```
// ANTES (en Color.kt, Theme.kt, Type.kt):
package com.example.compose

// DESPUES (en los 3 archivos):
package com.example.sanmarcosstore.ui.theme
```

2.4.3 Reemplazo de Archivos en el Proyecto

En Android Studio, navegar a `app/src/main/java/com/example/sanmarcosstore/ui/theme/`:

1. Borrar `Color.kt` del proyecto y arrastrar el `Color.kt` del ZIP.
2. Borrar `Theme.kt` del proyecto y arrastrar el `Theme.kt` del ZIP.
3. Borrar `Type.kt` del proyecto y arrastrar el `Type.kt` del ZIP.

2.4.4 Copia del Recurso `font_certs.xml`

1. Cambiar vista del Project Explorer de **Android** a **Project**.
2. Navegar a `app/src/main/res/`.
3. Si no existe la carpeta `values-v23`: click derecho en `res` → **New** → **Android Resource Directory** → Resource type: `values` → Qualifier: **Version 23**.
4. Arrastrar `font_certs.xml` del ZIP a `app/src/main/res/values-v23/`.

2.4.5 Error Comun: Unresolved reference R en Type.kt

Error Critico del Laboratorio

```
Type.kt:14:20 Unresolved reference 'R'.
```

El `Type.kt` del ZIP utiliza `R.array.com_google_android_gms_fonts_certs` pero no incluye el import de `R`. Tras cambiar el package, el simbolo no se resuelve automaticamente.

Solucion: agregar manualmente el import al inicio del archivo `Type.kt`.

Import a Agregar en Type.kt

```
package com.example.sanmarcosstore.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
// ... otros imports existentes ...
import com.example.sanmarcosstore.R    // <- AGREGAR ESTA LINEA
```

Truco rapido: posicionar el cursor sobre la `R` subrayada en rojo, presionar `Alt + Enter` (Windows/Linux) u `Option + Enter` (Mac), elegir **Import** y seleccionar `com.example.sanmarcosstore.R`.

2.4.6 Funcion AppTheme

El `Theme.kt` del ZIP define una funcion `AppTheme` estandar. Esta debe mantenerse con ese nombre, sin renombrarla. El alcance ya esta limitado por el package.

Funcion AppTheme en Theme.kt

```
@Composable
fun AppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    // ... contenido generado por Theme Builder ...
}
```

2.5 Test Inicial: Hola SanMarcosStore

Antes de armar la arquitectura modular, validar que el tema funciona. Reemplazar `MainActivity.kt` con el siguiente codigo de prueba:

MainActivity.kt - Version de Prueba

```
package com.example.sanmarcosstore

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.sanmarcosstore.ui.theme.AppTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(dynamicColor = false) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    Text(
                        text = "Hola SanMarcosStore!",
                        style = MaterialTheme.typography.headlineMedium,
                        color = MaterialTheme.colorScheme.primary,
                        modifier = Modifier.padding(24.dp)
                    )
                }
            }
        }
    }
}
```

Sobre `dynamicColor = false`: si se mantiene en `true`, Android 12+ ignora la paleta personalizada del Theme Builder y usa los colores del wallpaper del usuario. Para visualizar el tema generado, se desactiva.

2.6 Adaptacion para Otros Packages

Si el proyecto no se llama exactamente `SanMarcosStore`, todos los imports deben adaptarse al package real del proyecto.

Ejemplo con otro Package

```
// Si el proyecto usa el package com.illareklab.smstore:

import com.illareklab.smstore.ui.theme.AppTheme // en MainActivity.kt
import com.illareklab.smstore.R                 // en Type.kt
```

Regla general: todos los imports de la forma `com.example.sanmarcosstore.*` se reemplazan por `<PACKAGE_PROPIO>.*`. Los imports de `androidx.*` se mantienen sin modificar.

3 Arquitectura Modular del Proyecto

3.1 Estructura de Carpetas

Estructura Final del Proyecto

```
com.example.sanmarcosstore/  
+- MainActivity.kt           (Entry point, ~15 lineas)  
+- model/  
|   +- Producto.kt         (Modelo de datos)  
+- ui/  
|   +- components/  
|   |   +- ProductoItem.kt (Componente reutilizable)  
|   +- screens/  
|   |   +- TiendaScreen.kt (Pantalla 1)  
|   |   +- PerfilScreen.kt (Pantalla 2)  
|   +- navigation/  
|   |   +- AppNavigation.kt (Router)  
|   +- theme/  
|   |   +- Color.kt  
|   |   +- Theme.kt  
|   |   +- Type.kt
```

3.2 Creacion de Packages

En el Project Explorer, click derecho sobre `com.example.sanmarcosstore`:

1. **New** → **Package** → `model`
2. **New** → **Package** → `ui.components`
3. **New** → **Package** → `ui.screens`
4. **New** → **Package** → `ui.navigation`

Nota: al escribir packages con notacion de punto (`ui.components`), Android Studio crea la jerarquia anidada automaticamente.

3.3 Modelo de Datos: `Producto.kt`

model/Producto.kt

```
package com.example.sanmarcosstore.model  
  
data class Producto(  
    val id: Int,  
    val nombre: String,  
    val precio: String,  
    val categoria: String,  
    val favorito: Boolean = false  
)  
  
val productosDePrueba = listOf(  
    Producto(1, "Cafe Premium 500g", "S/ 45.00", "Bebidas", true),  
    Producto(2, "Chocolate artesanal 70%", "S/ 28.00", "Dulces"),  
    Producto(3, "Pan de masa madre", "S/ 18.00", "Panaderia"),  
    Producto(4, "Miel de abeja organica", "S/ 35.00", "Endulzantes", true),  
    Producto(5, "Mermelada de fresa", "S/ 22.00", "Conservas"),  
    Producto(6, "Te verde matcha", "S/ 55.00", "Bebidas"),  
    Producto(7, "Galletas de avena", "S/ 15.00", "Dulces"),  
    Producto(8, "Aceite de oliva extra virgen", "S/ 68.00", "Aceites")  
)
```

3.4 Componente Reutilizable: ProductoItem.kt

ui/components/ProductoItem.kt

```
package com.example.sanmarcosstore.ui.components

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.size
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Favorite
import androidx.compose.material.icons.filled.ShoppingCart
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.sanmarcosstore.model.Producto

@Composable
fun ProductoItem(producto: Producto) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceContainerLow
        ),
        elevation = CardDefaults.cardElevation(defaultElevation = 1.dp)
    ) {
        ListItem(
            headlineContent = {
                Text(producto.nombre,
                    style = MaterialTheme.typography.titleMedium)
            },
            supportingContent = {
                Text(producto.categoria,
                    style = MaterialTheme.typography.bodySmall)
            },
            trailingContent = {
                Column(horizontalAlignment = Alignment.End) {
                    Text(
                        producto.precio,
                        style = MaterialTheme.typography.labelLarge,
                        color = MaterialTheme.colorScheme.primary
                    )
                    if (producto.favorito) {
                        Icon(
                            Icons.Filled.Favorite,
                            contentDescription = "Producto favorito",
                            tint = MaterialTheme.colorScheme.tertiary,
                            modifier = Modifier.size(20.dp)
                        )
                    }
                }
            },
            leadingContent = {
                Icon(
                    Icons.Filled.ShoppingCart,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary
                )
            },
            colors = ListItemDefaults.colors(
                containerColor = MaterialTheme.colorScheme.surfaceContainerLow
            )
        )
    }
}
```

3.5 Pantalla Principal: TiendaScreen.kt

La pantalla Tienda incluye buscador con filtro en vivo, FAB que muestra/oculta un formulario con los 5 tipos de botones Material 3, y una lista filtrada de productos.

```

package com.example.sanmarcosstore.ui.screens

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Error
import androidx.compose.material.icons.filled.Search
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.sanmarcosstore.model.productosDePrueba
import com.example.sanmarcosstore.ui.components.ProductoItem

@Composable
fun TiendaScreen() {
    var busqueda by remember { mutableStateOf("") }
    var mostrarFormulario by remember { mutableStateOf(false) }

    Box(modifier = Modifier.fillMaxSize()) {
        Column(modifier = Modifier.fillMaxSize()) {
            OutlinedTextField(
                value = busqueda,
                onValueChange = { busqueda = it },
                label = { Text("Buscar producto") },
                placeholder = { Text("Ej: Cafe, Chocolate...") },
                leadingIcon = {
                    Icon(Icons.Filled.Search, contentDescription = null)
                },
                supportingText = { Text("Escribe para filtrar") },
                singleLine = true,
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
            )

            if (mostrarFormulario) {
                FormularioRapido()
            }

            val productosFiltrados = productosDePrueba.filter {
                it.nombre.contains(busqueda, ignoreCase = true) ||
                it.categoria.contains(busqueda, ignoreCase = true)
            }
        }
    }
}

```



```

        if (productosFiltrados.isEmpty()) {
            Box(
                modifier = Modifier.fillMaxSize(),
                contentAlignment = Alignment.Center
            ) {
                Text(
                    "No se encontraron productos",
                    style = MaterialTheme.typography.bodyLarge,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        } else {
            LazyColumn(
                contentPadding = PaddingValues(
                    start = 16.dp, end = 16.dp,
                    top = 8.dp, bottom = 88.dp
                ),
                verticalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                items(productosFiltrados, key = { it.id }) { producto ->
                    ProductoItem(producto = producto)
                }
            }
        }
    }

    FloatingActionButton(
        onClick = { mostrarFormulario = !mostrarFormulario },
        containerColor = MaterialTheme.colorScheme.primary,
        contentColor = MaterialTheme.colorScheme.onPrimary,
        modifier = Modifier
            .align(Alignment.BottomEnd)
            .padding(16.dp)
    ) {
        Icon(Icons.Filled.Add, contentDescription = "Agregar producto")
    }
}

```

```

@Composable
private fun FormularioRapido() {
    var nombreProducto by remember { mutableStateOf("") }
    val esError = nombreProducto.length > 30

    Card(
        modifier = Modifier.fillMaxWidth().padding(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceContainer
        )
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Agregar producto rapido",
                style = MaterialTheme.typography.titleMedium)
            Spacer(modifier = Modifier.height(8.dp))

            OutlinedTextField(
                value = nombreProducto,
                onValueChange = { nombreProducto = it },
                label = { Text("Nombre del producto") },
                supportingText = {
                    if (esError) Text("Maximo 30 caracteres",
                        color = MaterialTheme.colorScheme.error)
                    else Text("${nombreProducto.length}/30")
                },
                trailingIcon = {
                    if (esError) Icon(Icons.Filled.Error,
                        contentDescription = "Error",
                        tint = MaterialTheme.colorScheme.error)
                },
                isError = esError, singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )
            Spacer(modifier = Modifier.height(12.dp))

            // Los 5 tipos de boton Material 3
            Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                Button(onClick = {}, modifier = Modifier.weight(1f)) {
                    Text("Guardar")
                }
                FilledTonalButton(onClick = {}, modifier = Modifier.weight(1f)) {
                    Text("Borrador")
                }
            }
            Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                ElevatedButton(onClick = {}, modifier = Modifier.weight(1f)) {
                    Text("Vista previa")
                }
                OutlinedButton(onClick = {}, modifier = Modifier.weight(1f)) {
                    Text("Limpiar")
                }
            }
            TextButton(onClick = {}) { Text("Cancelar") }
        }
    }
}

```

3.6 Pantalla de Perfil: PerfilScreen.kt

La pantalla Perfil incluye cabecera con avatar circular, estadísticas en cards de color tertiary, lista de opciones del menu y switches para preferencias.

ui/screens/PerfilScreen.kt - Parte 1 (Imports y Modelo)

```
package com.example.sanmarcosstore.ui.screens

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ExitToApp
import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.unit.dp

data class OpcionPerfil(
    val id: Int,
    val titulo: String,
    val descripcion: String,
    val icono: ImageVector
)

private val opcionesPerfil = listOf(
    OpcionPerfil(1, "Mis pedidos", "Historial", Icons.Filled.Receipt),
    OpcionPerfil(2, "Mis favoritos", "Productos guardados", Icons.Filled.Favorite),
    OpcionPerfil(3, "Direcciones", "Envios", Icons.Filled.LocationOn),
    OpcionPerfil(4, "Configuracion", "Ajustes", Icons.Filled.Settings)
)
```

```

@Composable
fun PerfilScreen() {
    var modoOscuro by remember { mutableStateOf(false) }
    var notificaciones by remember { mutableStateOf(true) }

    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        // Cabecera con avatar
        item {
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer
                )
            ) {
                Row(
                    modifier = Modifier.fillMaxWidth().padding(20.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Box(
                        modifier = Modifier.size(72.dp)
                            .clip(CircleShape)
                            .background(MaterialTheme.colorScheme.primary),
                        contentAlignment = Alignment.Center
                    ) {
                        Icon(Icons.Filled.Person,
                            contentDescription = "Foto de perfil",
                            tint = MaterialTheme.colorScheme.onPrimary,
                            modifier = Modifier.size(40.dp))
                    }
                    Spacer(modifier = Modifier.width(16.dp))
                    Column {
                        Text("Ana Martinez",
                            style = MaterialTheme.typography.titleLarge,
                            color = MaterialTheme.colorScheme.onPrimaryContainer)
                        Text("ana.martinez@correo.com",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onPrimaryContainer)
                        Text("Cliente desde 2024",
                            style = MaterialTheme.typography.labelSmall,
                            color = MaterialTheme.colorScheme.onPrimaryContainer)
                    }
                }
            }
        }
    }
}

```

```

// Estadísticas
item {
    Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        EstadisticaCard("12", "Pedidos", Modifier.weight(1f))
        EstadisticaCard("5", "Favoritos", Modifier.weight(1f))
        EstadisticaCard("3", "Cupones", Modifier.weight(1f))
    }
}

item { Text("Mi cuenta",
    style = MaterialTheme.typography.titleMedium) }

items(opcionesPerfil, key = { it.id }) { opcion ->
    Card(
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceContainerLow
        )
    ) {
        ListItem(
            headlineContent = { Text(opcion.titulo) },
            supportingContent = { Text(opcion.descripcion) },
            leadingContent = { Icon(opcion.icono,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary) },
            trailingContent = { Icon(
                Icons.AutoMirrored.Filled.KeyboardArrowRight,
                contentDescription = "Ir a ${opcion.titulo}",
                tint = MaterialTheme.colorScheme.onSurfaceVariant) },
            colors = ListItemDefaults.colors(
                containerColor = MaterialTheme.colorScheme.surfaceContainerLow
            )
        )
    }
}

```

```

        item { Text("Preferencias",
            style = MaterialTheme.typography.titleMedium) }

        item {
            Card(colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.surfaceContainerLow
            ) {
                Column {
                    ListItem(
                        headlineContent = { Text("Modo oscuro") },
                        supportingContent = { Text("Cambia el aspecto") },
                        trailingContent = {
                            Switch(checked = modoOscuro,
                                onCheckedChange = { modoOscuro = it })
                        }
                    )
                    HorizontalDivider()
                    ListItem(
                        headlineContent = { Text("Notificaciones") },
                        supportingContent = { Text("Alertas de promociones") },
                        trailingContent = {
                            Switch(checked = notificaciones,
                                onCheckedChange = { notificaciones = it })
                        }
                    )
                }
            }
        }

        item {
            OutlinedButton(
                onClick = { /* cerrar sesion */ },
                modifier = Modifier.fillMaxWidth(),
                colors = ButtonDefaults.outlinedButtonColors(
                    contentColor = MaterialTheme.colorScheme.error
                ) {
                    Icon(Icons.AutoMirrored.Filled.ExitToApp, null)
                    Spacer(modifier = Modifier.width(8.dp))
                    Text("Cerrar sesion")
                }
            )
        }
    }
}

@Composable
private fun EstadisticaCard(valor: String, etiqueta: String,
    modifier: Modifier = Modifier) {
    Card(modifier = modifier, colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.tertiaryContainer)) {
        Column(modifier = Modifier.fillMaxWidth().padding(12.dp),
            horizontalAlignment = Alignment.CenterHorizontally) {
            Text(valor, style = MaterialTheme.typography.headlineMedium,
                color = MaterialTheme.colorScheme.onTertiaryContainer)
            Text(etiqueta, style = MaterialTheme.typography.labelMedium,
                color = MaterialTheme.colorScheme.onTertiaryContainer)
        }
    }
}

```

3.7 Router de Navegacion: AppNavigation.kt

ui/navigation/AppNavigation.kt - Parte 1

```
package com.example.sanmarcosstore.ui.navigation

import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Store
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.navigation.NavDestination.Companion.hierarchy
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import com.example.sanmarcosstore.ui.screens.PerfilScreen
import com.example.sanmarcosstore.ui.screens.TiendaScreen

sealed class Ruta(
    val ruta: String,
    val etiqueta: String,
    val icono: ImageVector
) {
    data object Tienda : Ruta("tienda", "Tienda", Icons.Filled.Store)
    data object Perfil : Ruta("perfil", "Mi Perfil", Icons.Filled.Person)
}

private val pestañas = listOf(Ruta.Tienda, Ruta.Perfil)
```

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AppNavigation() {
    val navController = rememberNavController()
    val currentBackStack by navController.currentBackStackEntryAsState()
    val currentDestination = currentBackStack?.destination

    val tituloActual = when (currentDestination?.route) {
        Ruta.Tienda.ruta -> "SanMarcos Store"
        Ruta.Perfil.ruta -> "Mi Perfil"
        else -> "SanMarcos Store"
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(tituloActual) },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer,
                    titleContentColor = MaterialTheme.colorScheme.onPrimaryContainer
                )
            )
        },
        bottomBar = {
            NavigationBar(
                containerColor = MaterialTheme.colorScheme.surfaceContainer
            ) {
                pestañas.forEach { pestaña ->
                    val seleccionada = currentDestination
                        ?.hierarchy?.any { it.route == pestaña.ruta } == true

                    NavigationBarItem(
                        selected = seleccionada,
                        onClick = {
                            navController.navigate(pestaña.ruta) {
                                popUpTo(navController.graph.startDestinationId) {
                                    saveState = true
                                }
                                launchSingleTop = true
                                restoreState = true
                            }
                        },
                        icon = { Icon(pestaña.icono,
                            contentDescription = pestaña.etiqueta) },
                        label = { Text(pestaña.etiqueta) }
                    )
                }
            }
        }
    ) { padding ->
        NavHost(
            navController = navController,
            startDestination = Ruta.Tienda.ruta,
            modifier = Modifier.padding(padding)
        ) {
            composable(Ruta.Tienda.ruta) { TiendaScreen() }
            composable(Ruta.Perfil.ruta) { PerfilScreen() }
        }
    }
}

```

Patron popUpTo + launchSingleTop + restoreState: estas tres banderas conforman el patron estandar de bottom navigation. Evitan acumulacion de pantallas en el back stack, mantienen una sola instancia por tab y preservan el estado de cada pantalla al cambiar entre tabs.

3.8 MainActivity Final

Tras crear todos los modulos, MainActivity.kt se reduce a su minima expresion: aplicar el tema y delegar la navegacion.

MainActivity.kt - Version Final

```
package com.example.sanmarcosstore

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import com.example.sanmarcosstore.ui.navigation.AppNavigation
import com.example.sanmarcosstore.ui.theme.AppTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(dynamicColor = false) {
                AppNavigation()
            }
        }
    }
}
```

4 Diagrama de Flujo

Pipeline de Renderizado de la App

```
[ Usuario abre la app ]
      |
      v
[ MainActivity ] -> [ AppTheme aplica colores ]
      |
      v
[ AppNavigation ] -> [ Scaffold con TopBar + BottomBar ]
      |
      v
+---+---+
|           |
|           |
v           v
[ Tienda ]  [ Perfil ]
|           |
v           v
[ LazyColumn ] [ LazyColumn ]
|           |
v           v
[ ProductoItem ] [ Cards de perfil ]
```

5 Solucion de Problemas

Error	Solucion
Type.kt:14 Unresolved reference 'R'	Falta el import com.example.sanmarcosstore.R en Type.kt. Usar Alt+Enter sobre la R subrayada para importar automaticamente
Unresolved reference: AppTheme	Falta el import en MainActivity. Verificar: import com.example.sanmarcosstore.ui.theme.AppTheme
Unresolved reference: primaryLight	Los packages de Color.kt, Theme.kt y Type.kt no coinciden. Los tres deben ser com.example.sanmarcosstore.ui.theme
Unresolved reference: Icons.Filled.Search	Falta la dependencia de iconos extendidos. Agregar material-icons-extended:1.7.8
Unresolved reference: GoogleFont	Falta la dependencia. Agregar ui-text-google-fonts:1.7.8

Resource not found: com.google.android.gms_fonts	No se copio font_certs.xml. Verificar que este en res/values-v23/
Colores azules en lugar de los del tema	dynamicColor = true en Android 12+. Cambiar a false en MainActivity
NavigationBar no se renderiza	Falta la dependencia navigation-compose:2.8.0. Verificar y sincronizar Gradle
data object no compila	Version vieja de Kotlin. Cambiar data object por object en AppNavigation.kt
Falta @OptIn(ExperimentalMaterial3Api)	Componentes experimentales (TopAppBar). Agregar la anotacion arriba del Composable
Crash al iniciar la app	Falta el import de algun Composable. Revisar Logcat para detalle del error
Compilacion falla repetidamente	Cache corrupto. File → Invalidate Caches → Invalidate and Restart

6 Ejecucion y Verificacion

6.1 Comportamiento Esperado

Validacion Funcional Post-Compilacion

Accion	Resultado esperado
Abrir la app	Pantalla Tienda con lista de productos sobre fondo del tema
Tap en FAB (+)	Aparece/desaparece formulario con los 5 botones
Escribir en buscador	Lista se filtra en tiempo real
Tap en tab "Mi Perfil"	Cambia a pantalla de perfil con avatar y opciones
Toggle de switches	Cambian visualmente al instante
Volver al tab "Tienda"	La busqueda escrita previamente se conserva

6.2 Configuracion del Emulador

1. **Tools** → **Device Manager** → **Create Virtual Device**
2. Hardware recomendado: **Pixel 7**
3. System Image: **API 34 (Android 14)** - eleccion balanceada entre features modernas y compatibilidad
4. Click **Finish** y esperar a la primera carga del AVD

7 Consideraciones de Implementacion

Limitaciones y Buenas Practicas

- ✓ **Permite:** Arquitectura escalable a 10+ pantallas con minimo refactor
- ✓ **Permite:** Tema dinamico (Material You) en Android 12+ activando `dynamicColor = true`
- ✓ **Permite:** Reutilizar `ProductoItem` en cualquier otra pantalla del proyecto
- ✗ **No incluye:** Persistencia de datos (productos en memoria, se pierden al cerrar)
- ✗ **No incluye:** Validacion de campos avanzada ni red (API REST)
- † **Advertencia:** El `material-icons-extended` agrega 5-7 MB al APK. Para produccion considerar usar solo los iconos necesarios via `Vector Drawables`
- **Solucion:** Para apps en produccion, implementar `Room` (persistencia) y `Retrofit` (red) en sesiones posteriores

8 Ejercicios Sugeridos para Entrega

8.1 Nivel 1: Personalizacion

1. Cambiar el color seed en Material Theme Builder por uno distinto (verde, azul, naranja). Reemplazar solo `Color.kt` y documentar el cambio visual.
2. Personalizar los datos del usuario en `PerfilScreen` con datos reales del estudiante.

8.2 Nivel 2: Funcionalidad

3. Agregar un tercer tab `Carrito` con icono `ShoppingCart`. Mostrar al menos una `Card` con mensaje "Tu carrito esta vacio".
4. Implementar funcionalidad clickable en el icono de favorito (corazon) de `ProductoItem` usando `mutableStateListOf`.
5. Agregar filtros por categoria usando `FilterChip` arriba de la lista en `TiendaScreen`.

8.3 Nivel 3: Avanzado

6. Al hacer click en un `ProductoItem`, navegar a una pantalla de detalle que reciba el ID del producto como parametro de ruta.
7. Implementar modo oscuro real: el `Switch` del perfil cambia el tema completo de la app (usar `CompositionLocalProvider` o `ViewModel` compartido).
8. Persistir el estado de favoritos usando `DataStore` para que sobrevivan al cierre de la app.

9 Entregable

Checklist de Entrega en GitHub

1. Repositorio en GitHub con el código completo (excluyendo build/, .gradle/, .idea/)
2. Capturas de pantalla:
 - Pantalla Tienda con productos visibles
 - Pantalla Tienda con formulario abierto (5 botones)
 - Pantalla Mi Perfil completa (scroll)
3. README.md con: nombre, descripción breve, captura principal, ejercicios completados (Nivel 1/2/3)

10 Referencias y Recursos

- [Material Design 3 - Documentación oficial](#)
- [Material Theme Builder](#)
- [Jetpack Compose - Android Developers](#)
- [Navigation Compose Guide](#)
- [Compose Components Catalog](#)
- [Compose Samples \(GitHub oficial\)](#)
- [Material Symbols - Catálogo de iconos](#)

Doc ID: CODLAB-SMSTORE-2026-003 | Version: 1.0 | Status: Production Ready

Este documento es parte del protocolo estándar de desarrollo Android moderno de Illarek-Lab